ALGORITHMS PROJECT 3 (BALANCED STRING)

Prepared by

Abdelrahman Ahmed

Abdelrahman Hasan

Ammar Salah

Moaz Ahmed

Yehia Ahmed

Ahmed Salah

Youssef Ahmed

NON RECURSIVE APPROACH

Pseudocode:

INPUT: A string s of lowercase letters

OUTPUT: Length of the longest substring containing exactly 2 distinct characters with equal frequencies

```
BEGIN
n = LENGTH(s)
maxLen = 0
FOR i = 0 TO n - 1
    count[0..25] = {0}
    FOR j = i TO n - 1 DO
        count[s[j] - 'a'] = count[s[j] - 'a'] + 1
         distinct = 0
```

```
        first = -1
        second = -1
        FOR k = 0 TO 25
            IF count[k] > 0 THEN
                distinct = distinct + 1
                IF first = -1 THEN
                    first = count[k]
                ELSE
                    second = count[k]
                END IF
            END IF
        END FOR
        IF distinct = 2 AND first = second THEN
            len = j - i + 1
            IF len > maxLen THEN
                maxLen = len
            END IF
        END IF
    END FOR
END FOR
RETURN maxLen
END
```

Implementation:

```
#include <iostream>
#include <string>
using namespace std;

int longestBalancedSubstring(const string &s) {
```

```
int n = s.length();
int maxLength = 0;

for (int i = 0; i < n; i++) {
   int count[26] = {0};

   for (int j = i; j < n; j++) {
      count[s[j] - 'a']++;

      int distinct = 0;
      int first = -1, second = -1;

      for (int k = 0; k < 26; k++) {
         if (count[k] > 0) {
            distinct++;
            if (first == -1) first = count[k];
            else second = count[k];
         }
      }

      if (distinct == 2 && first == second) {
         int len = j - i + 1;
         if (len > maxLength)
            maxLength = len;
      }
   }
```

```
    }

    return maxLength;
}

int main() {
    string s;
    cin >> s;

    cout << longestBalancedSubstring(s);

    return 0;
}
```

Analysis:

1. Algorithmic Approach:

The implemented solution follows a Non-Recursive (Iterative) approach. It utilizes nested loops to explore all possible substrings and an auxiliary frequency array to validate the balanced condition.

2.Time complexity Analysis:

Component	Logic	Complexity
Outer Loop (i)	Iterates through the string from index 0 to *n-1*.	`O(n)`
Inner Loop (j)	Iterates from the current *i* to *n-1* to form substrings.	`O(n)`
Validation Loop (k)	Iterates through a fixed frequency array of 26 characters.	`O(1)`

Total Time Complexity: $O(n^2)$ Since the algorithm uses two nested loops where the number of operations scales quadratically with the input size n, the complexity is $O(n^2)$. The innermost loop is constant (26 iterations) and does not depend on n.

3. Space Complexity Analysis:

The space complexity measures the extra memory (Auxiliary Space) required by the algorithm:

• Frequency Array: An integer array count[26] is used. Its size is fixed regardless of n. Variables: Primitive types (int) are used for indexing and length tracking. Total Space Complexity: O(1) (Constant Space)

4. Summary Table:

Scenario	Complexity
Best Case	$\Omega(n^2)$
Average Case	$\Theta(n^2)$
Worst Case	$O(n^2)$
Auxiliary Space	O(1)

Recursive

PSEUDOCODE: Longest Balanced Substring

Goal:

Find the longest substring that contains exactly 2 distinct characters

with equal frequencies.

Example balanced substrings:

"aabb", "abab", "xyxy"

```
FUNCTION isBalance(String, start, end):

    array frequency[26] = {0}

    different_chars = 0


    FOR i FROM start TO end:

        index = String[i] - 'a'


        IF frequency[index] == 0 THEN:

            different_chars = different_chars + 1

        END IF

        frequency[index] = frequency[index] + 1

    END FOR


    IF different_chars != 2 THEN:

        RETURN false
```

```
END IF


value1 = 0

value2 = 0


FOR i FROM 0 TO 25:

   IF frequency[i] > 0 THEN:

      IF value1 == 0 THEN:

         value1 = frequency[i]

      ELSE:

         value2 = frequency[i]

         BREAK


      END IF
```

```
    END IF

  END FOR

  RETURN (value1 == value2)

END FUNCTION

-------------------------------------------------

FUNCTION startFunction(String, i):

  IF i == Length(String) THEN:

    RETURN 0

  END IF

  option1 = endFunction(String, i, i)

  option2 = startFunction(String, i + 1)
```

```
    RETURN max(option1, option2)

END FUNCTION

-------------------------------------------------



FUNCTION endFunction(String, i, j):

    IF j == Length(String) THEN:

        RETURN 0

    END IF



    currentLength = 0



    IF isBalance(String, i, j) THEN:

        currentLength = j - i + 1
```

```
    END IF

    RETURN max(currentLength, endFunction(String, i, j + 1))

END FUNCTION

-------------------------------------------------

FUNCTION longestBalanced(String):

    RETURN startFunction(String, 0)

END FUNCTION

-------------------------------------------------

MAIN:

    INPUT String
```

```
    result = longestBalanced(String)

    OUTPUT "Longest Balanced Length: " + result

END MAIN
```

Implementation:

```
#include <iostream>
#include <string>
#include <algorithm>

using namespace std;

int endFunction(const string& s, int i, int j);

bool isBalance(const string& s, int start, int end) {
    int frequency[26] = {0};
    int different_chars = 0;

    for (int i = start; i <= end; i++) {
        int index = s[i] - 'a';

        if (frequency[index] == 0) {
            different_chars = different_chars + 1;
        }
```

```
        frequency[index] = frequency[index] + 1;
    }

    if (different_chars != 2) {
        return false;
    }

    int value1 = 0;
    int value2 = 0;

    for (int i = 0; i < 26; i++) {
        if (frequency[i] > 0) {
            if (value1 == 0) {
                value1 = frequency[i];
            } else {
                value2 = frequency[i];
                break;
            }
        }
    }

    return value1 == value2;
}

int startFunction(const string& s, int i) {
```

```
    if (i == s.length()) {
        return 0;
    }

    int option1 = endFunction(s, i, i);
    int option2 = startFunction(s, i + 1);

    return max(option1, option2);
}

int endFunction(const string& s, int i, int j) {
    if (j == s.length()) {
        return 0;
    }

    int currentLength = 0;

    if (isBalance(s, i, j)) {
        currentLength = j - i + 1;
    }

    return max(currentLength, endFunction(s, i, j + 1));
}

int longestBalanced(const string& s) {
    return startFunction(s, 0);
```

```
}

int main() {
    string input;

    cout << "Enter string: ";
    cin >> input;

    int result = longestBalanced(input);

    cout << "Longest Balanced Length: " << result << endl;

    return 0;
}
```

Analysis:

1. Algorithmic Approach:

The implemented solution follows a Recursive approach. It uses recursive functions to explore all possible substrings of the input string. The `startFunction` determines the starting position of each substring, while the `endFunction` determines its ending position. For every generated substring, the `isBalance` function checks whether it contains exactly two distinct characters with equal frequencies.

2. Time complexity Analysis:

Component	Logic	Complexity
startFunction	Recursively iterates through all possible starting positions.	O(n)

endFunction	Recursively iterates through all possible ending positions for each start.	O(n)
IsBalance	Scans the current substring to count its characters.	O(n)

Total Time Complexity: $O(n^3)$

Since the algorithm generates all possible substrings using two recursive levels, this contributes $O(n^2)$. For each substring, the `isBalance` function scans up to n characters, which adds another $O(n)$ factor. Therefore, the overall time complexity is:

O(n)×O(n)×O(n)=O(n3)

3. Space Complexity Analysis

The space complexity measures the extra memory required by the algorithm:

- **Frequency Array:** An integer array `frequency[26]` is used inside `isBalance`. Its size is fixed regardless of n.
- **Recursion Stack:** The recursive calls of `startFunction` and `endFunction` require stack memory. The maximum depth grows with the input size.

Total Space Complexity: $O(n)$

The overall memory usage is dominated by the recursion stack, while the frequency array remains constant in size.

Scenario	Complexity
Best case	$\Omega(n^3)$
Average case	$\Theta(n^3)$
Worst case	$O(n^3)$
Auxiliary Space	O(n)

NOW IT'S TIME FOR COMPARISONS

1.pseudo vs pseudo

Comparison Point	Non -recursive	Recursive
Main Techniques	Uses nested loops to generate all possible substrings.	Uses recursive functions to generate all possible substrings.
Substring Generation	The start and end of each substring are controlled by `i` and `j` loops.	The start and end of each substring are controlled by `startFunction` and `endFunction`.
Balance Checking	The frequency array is updated gradually while the substring expands.	Each substring is checked separately by calling `isBalance`.
Code Structure	The logic is mainly written inside one algorithm.	The logic is divided into multiple functions.
Efficiency	More efficient because it does not recount the whole substring each time.	Less efficient because it scans each substring again during checking.

2.Non Recursive VS Recursive Imp

Comparison Point	Non -recursive	Recursive
Main Technique	uses nested `for` loops to examine all possible substrings.	Uses recursive function calls to examine all possible substrings.
Substring Generation	The variables `i` and `j` control the starting and ending positions.	`startFunction` controls the start position, while `endFunction` controls the end position.
Balance checking	Character frequencies are updated while the substring expands.	The `isBalance` function scans each substring separately.
Code organization	Most of the logic is written inside one main function.	The logic is divided into several helper functions.
Memory usage	Uses only fixed extra memory without recursion	Uses extra memory because of the recursion stack.
Practical Efficiency	Faster and more suitable for execution	Slower because it performs more repeated checks.

3.Recursive vs Non Recursive Analysis

Comparison Point	Non-Recursive	Recursive
Time complexity	$O(n^2)$	$O(n^3)$
Reason for time complexity	It uses two nested loops, while the validation loop checks only 26 characters, so it is considered constant.	It generates all substrings recursively, and for each substring, `isBalance` scans the substring again.
Space complexity	O(1)	O(n)
Reason for space complexity	It uses a fixed frequency array of size 26 and a few simple variables.	It uses a fixed frequency array, but the recursion stack grows with the input size.
Overall efficiency	More efficient in both time and memory.	Less efficient because it requires more repeated scanning and extra recursive memory.

4.Non recursive Vs recursive approach

Comparison Point	Non-Recursive	Recursive
Execution Method	Solves the problem using iterative loops.	Solves the problem using recursive function calls.
Substring Exploration	Generates substrings directly using loop variables.	Generates substrings through repeated recursive calls.
Performance	Faster because it avoids rescanning every substring from the beginning.	Slower because each substring is checked separately from scratch.
Memory Usage	Requires constant extra space.	Requires extra memory for the recursion stack.
Suitability	More suitable as the efficient algorithm for this problem.	Useful for demonstrating recursion, but less efficient in practice.

DONE& THANK U